目錄

1 EShop 功能簡介： 5

2 EShop 的架構： 5

2.1 Entity： 5

2.1.1 Entity 彼此的關聯方式： 5

2.2 後端架構： 5

2.2.1 後端功能： 5

2.2.2 前端功能： 6

3 專案建置流程： 6

3.1 Spring 設定 xml 檔+Spring 設定 Java 檔： 6

3.2 Struts 設定檔： 6

3.2.1 預設最基本的 Struts 設定檔： 6

3.2.2 客製化最基本的 Struts 設定檔： 7

3.2.3 其他的 Struts 設定檔，必須被 include 進 struts.xml： 8

3.2.4 設定全域 redirect 方式： 8

3.3 web.xml： 8

3.3.1 初始化 Spring： 8

3.3.2 初始化 Struts 2： 9

4 EShop 的註冊功能： 9

4.1 註冊頁面： 9

4.2 進入後端： 10

4.2.1 struts-register.xml： 10

4.2.2 Action： 10

4.2.3 Service： 11
4.2.4 回到 Action： 11
4.3 呈顯註冊成功訊息於前端頁面： 11
4.4 註冊成功畫面： 12
5 EShop 的登入功能： 12
5.1 登入頁面： 12
5.2 提交到後端： 14
5.2.1 設置 url 與 Struts 2 Action 的具體方法，之間綁定的方式： 14
5.2.2 提交給 Struts 2 Action，將登入資訊寫入 VO、session： .. 14
5.2.3 登入過程中，輸入的密碼，如何與已經經過雜湊+加鹽的密碼來做比較： 15
5.2.4 forward 到登入成功後，要進入的頁面： 15
6 EShop 首頁功能： 16
6.1 首頁頁面概說： 16
6.2 前端頁面： 17
6.3 後端渲染資料： 18
7 商品 CRUD 功能： 19
7.1 商品查詢功能： 19
7.1.1 於 index.jsp 點擊連結： 19
7.1.2 action： 19
7.1.3 ProductService： 20
7.1.4 ProductDaoImpl： 21
7.1.5 forward 到 product_list.jsp： 21
7.1.6 查詢畫面： 22

8 購物車 CRUD 功能： 23
8.1 新增商品至購物車之功能： 23
8.1.1 於 product_list.jsp，新增商品之按鈕： 23
8.1.2 struts-product.xml，Struts 2 url mapping： 23
8.1.3 依序進入 action、service、dao： 23
8.1.4 商品明細頁面： 24
8.1.5 將商品加入購物車： 27
8.1.6 實際頁面： 30
8.2 進入購物車頁面： 32
8.2.1 按下該按鈕後，根據按鈕的 href，進行 Struts 2 url mapping： 32
8.2.2 將購物車表單與 Action 內的屬性綁定： 32
8.2.3 實際頁面： 35
9 付款/訂單 CRUD 功能： 35
9.1 checkPurchase.jsp： 35
9.2 表單 action 對 Struts 2 url mapping： 36
9.3 進入對應 action(PurchaseAction)之方法(purchase)： 37
9.4 付款完成，即將跳回專案頁面之前的銜接頁面(可以在這裡，設定付款完成後要做的動作)： 38
9.5 實際頁面： 39
9.6 付款完成後動作： 39
9.6.1 將訂單狀態改為「已付款」： 40
9.6.2 呈顯付款完成時，真正要呈顯的 jsp 頁面： 41
9.7 付款成功頁面： 42
9.7.1 「查看訂單」鈕： 42

9.8 訂單清單頁面： 43
9.9 更新版訂單清單頁面(以前端方式添加分頁功能)： 44
9.10 更新版訂單清單頁面(以後端方式添加分頁功能)： 46
9.10.1 DAO： 46
9.10.2 Service： 46
9.10.3 Action： 46
9.10.4 瀏覽器 URL： 47
10 管理者端頁面，處理訂單功能： 47
10.1 DAO： 47
10.2 Service： 47
10.3 Action： 48
10.4 struts-order.xml： 48
10.5 order_list_forAdmin.jsp： 48
10.5.1 呈顯訂單資料： 48
10.5.2 設置修改訂單狀態功能： 49
10.5.3 實際頁面： 52
11 錯誤處理： 53
11.1 出現 java.lang.NoSuchMethodException: com.sun.proxy.$Proxy83.doLogin() 但是 com.example.action.LoginAction 確實有 doLogin 方法。 53
11.2 debug 模式看到的物件值是 null，但是用 print 印出來又有值。 53

1 EShop 功能簡介：

EShop 是一個簡易的購物商城模擬系統。

2 EShop 的架構：

2.1 Entity：

既然是購物商城，那麼在 Entity 的部分，就會有商品、用戶、訂單、購物車，這四個主要的 Entity：
商品 Entity 主要的欄位：唯一可供辨識的 id、商品編號、名稱、價格。
用戶 Entity 主要的欄位：唯一可供辨識的 id、登入帳號名稱、密碼、真實姓名、電話、註冊日期、用戶類型(一般用戶或管理者)。
訂單主要的欄位：唯一可供辨識的 id、訂單編號、購買商品 id、商品的數量、購買用戶之 id。
購物車 Entity 主要的欄位：唯一可供辨識的 id、商品 id、用戶 id、商品數量。

2.1.1 Entity 彼此的關聯方式：

訂單與商品，是一對一的關係；
訂單與用戶的關係，是多對一的關係；
購物車對商品，是一對一的關係、對用戶的關係，是多對一的關係。

2.2 後端架構：

採用常見的 MVC 與 DAO-Service-Action/Controller 架構。
具體使用了 Struts 2+Hibernate+Spring 作為實作後端架構的技術。

2.2.1 後端功能：

登入功能
商品 CRUD 功能
用戶 CRUD 功能
購物車 CRUD 功能
訂單 CRUD 功能

2.2.2 前端功能：

用戶登入頁面。

模擬商城首頁。

商品列表。

購買商品頁面。

用戶購物車頁面。

確認付款頁面。

付款成功頁面。

用戶訂單清單頁面。

3 專案建置流程：

3.1 Spring 設定 xml 檔+Spring 設定 Java 檔：

```
@Configuration
@ImportResource("classpath:spring/applicationContext.xml")
@EnableAspectJAutoProxy
public class TestConfig {
}
```

為何不將全部設定，集中於 xml 檔，還要另外有 java 檔？因為要簡化 Spring AOP 的設定方式。

另外還需要用到 Spring Security 的對密碼做雜湊+加鹽的動作，故需註冊 BCryptPasswordEncoder：

```
@Bean
    public BCryptPasswordEncoder bCryptPasswordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

3.2 Struts 設定檔：

3.2.1 預設最基本的 Struts 設定檔：

預設最基本的 Struts 設定檔，是 struts-default.xml，這個 Struts 設定檔是專門給開發者繼承 package 的，以讓開發者可以實作自身最基本的 Struts 設定檔。

3.2.2 客製化最基本的 Struts 設定檔：

其在 Maven 專案，必須直接放在 src/main/resources 之下。因為 struts 2 啟動時，是固定到 classpath 找該檔案的。所以就不用明確標示 struts.xml 位置？因為只要將該檔置於 src/main/resources，Struts 2 便會自動找到。

3.2.2.1 繼承預設最基本的 Struts 設定檔：

```
<include file="struts-default.xml" />
```

要讓 struts 設定檔繼承 struts-default.xml 內的 package，也就是繼承既有的攔截器，就必須先 include struts-default.xml。

```
<package name="default">
```

用來設定客製化最基層的 package，來讓其他子 package 來繼承。
客製化最基層的 package，需要先繼承 struts-default package；故改寫成

```
<package name="default" extends="struts-default">
```

3.2.2.2 自訂攔截器與攔截器集合：

```
<interceptors>
    <interceptor name="authenticationInterceptor"
    class="com.example.interceptor.AuthenticationInterceptor" />

    <interceptor-stack name="globalStack">
        <interceptor-ref name="authenticationInterceptor" />
        <interceptor-ref name="fileUpload" />
        <interceptor-ref name="modelDrivenStack" />
    </interceptor-stack>

</interceptors>
<interceptor-stack name="globalStack">
```

表示宣告一個客製化攔截器集合。

```
<interceptor-ref name="modelDrivenStack" />
```

struts.xml 註冊 modelDrivenStack，表示宣告要使用這個攔截器集合 (modelDriven+basicStack)，凡是攔截到有 model 的 actionContext，其 model 都要推到 valueStack 頂端、且 model 可以被 basicStack 內的 parameters 攔截器賦值。
modelDriven、basicStack 攔截器，均來自 struts-default package。

3.2.2.3 設定 package 預設的攔截器集合：

```
<default-interceptor-ref name="globalStack" />
```

宣告該客製化攔截器集合，是該 package 預設的攔截器集合。

3.2.3 其他的 Struts 設定檔，必須被 include 進 struts.xml：

其他的 Struts 設定檔：

```
<include file="struts/struts-login.xml"/>
<!-- 這是相對於 classpath 的相對路徑 -->
```

為何上面的路徑，不是寫成/struts/struts-login.xml，因為這樣寫是絕對路徑，但現在是需要相對於 classpath 的相對路徑。

3.2.4 設定全域 redirect 方式：

```
<global-results>
    <result name="login" type="redirect">
        /login.jsp
    </result>
</global-results>
```

/login.jsp 這個 jsp 檔，位於 src/main/webapps 底下，那麼/login.jsp 開頭的「/」，是指這是個絕對路徑，表示 contextPath 開頭的絕對路徑。

靜態資源或著位於 WEB-INF 之外的檔案，如果不透過 struts 2 來取得，那是由伺服器直接處理，WEB-INF 內的 jsp 檔則必須透過 action 存取。

3.3 web.xml：

3.3.1 初始化 Spring：

```
<!-- spring 容器初始化-->
    <listener>
        <listener-
class>org.springframework.web.context.ContextLoaderListener</listener-
class>
    </listener>

    <!-- 引入 Spring 配置 java 檔的方式 -->
     <context-param>
```

```
        <param-name>contextClass</param-name>
        <param-
value>org.springframework.web.context.support.AnnotationConfigWebApplicatio
nContext</param-value>
    </context-param>

    <context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>com.example.TestConfig</param-value>
    </context-param>
```

3.3.2 初始化 Struts 2：

```
    <filter>
        <filter-name>struts2</filter-name>
        <filter-
class>org.apache.struts2.dispatcher.filter.StrutsPrepareAndExecuteFilter</f
ilter-class>
    </filter>
    <filter-mapping>
        <filter-name>struts2</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>
```

Struts 2 以 StrutsPrepareAndExecuteFilter 作為類似 dispatcherServlet 的分發器；請求會先到 StrutsprepareAndExecuteFilter，再到其他攔截器。

4 EShop 的註冊功能：

4.1 註冊頁面：

檔名：login.jsp

先添加一個註冊按鈕：

```
<button type="button" class="btn btn-success" id="registerButton">註冊
</button>
```

設定當該按鈕被 click 之時，呼叫提交表單到後端註冊方法之函數；且是在整個頁面 onload，該按鈕已被瀏覽器認知到之時，才做該設定：

```
onload=function(){
//$(document).ready(function(){})，不等於 onload=function(){}

    document.querySelector("#registerButton").addEventListener("click",func
tion(e){
                validateAndSubmit(e);
            });
validateAndSubmit(e)：
if(e.target.id=="registerButton"){
            $("#form").attr('action',
'<%=request.getContextPath()%>/register/save');
          }

          $("#form").submit();
```

4.2 進入後端：

4.2.1 struts-register.xml：

```
<package name="register" extends="default" namespace="/register">
<action name="save" class="com.example.action.RegisterAction"
method="save">
    <result name="success" type="redirect">/login.jsp</result>
</action>
```

4.2.2 Action：

在 Action 最重要的，就是要對用戶密碼，進行 hash+加鹽：

```
user.setPassword(bCryptPasswordEncoder.encode(request.getParameter("passwor
d")));
```

4.2.2.1 hash+加鹽的動作，是使用 Spring Security：

TestConfig：

```
@Bean
    public BCryptPasswordEncoder bCryptPasswordEncoder() {
```

```
        return new BCryptPasswordEncoder();
    }
```

Action 就可直接注入：

```
@Autowired
    private BCryptPasswordEncoder bCryptPasswordEncoder;
```

4.2.3 Service：

```
userRepository.save(user);
```

在 service，可能會有多個彼此關聯的 vo 需要被 save，這時需要 @Transactional

4.2.4 回到 Action：

4.2.4.1 註冊成功，要回傳註冊成功訊息到前端頁面：

```
getSession().setAttribute("registerResult", "註冊成功，請登入");
```

這裡不可用 request，因為 redirect 會導致 request 重新產生，進而使原本 request 的資料丟失。

4.2.4.2 搭配重導到登入頁面：

```
<result name="success" type="redirect">/login.jsp</result>
```

4.3 呈顯註冊成功訊息於前端頁面：

login.jsp：

```
<c:if test="${not empty registerResult}">
    <script>
        alert('<c:out value="${registerResult}" />');
    </script>
    <c:remove var="registerResult" scope="session"/>
</c:if>
```

```
<c:if test="${not empty registerResult}">
```

如果 c:if 是 false，那麼被包圍起來的程式，完全不會被輸出到瀏覽器。

```
<c:out value="${registerResult}" />
```

使用 JSTL 的 c:out 的原因：

避免 XSS：其會將 HTML Code，例如<、>、&，轉成純文字格式：

`<>&`，也就是自動 escape 特殊字元；這些符號，只有在要被呈顯出來時，會被瀏覽器轉成對應的純文字格式：<、>、&。
當遇到`${msg}`為 null 之時，會輸出空字串；而如果直接寫成`${msg}`，有可能直接在瀏覽器頁面上，顯示 null。

```
<c:remove var="registerResult" scope="session"/>
```

registerResult 放在 session 內，在註冊、登入完後，就移除掉。

4.4 註冊成功畫面：



5 EShop 的登入功能：

5.1 登入頁面：

login.jsp：

實現功能方式：以 bootstrap 為框架，搭配傳統 JavaScript 表單提交、前後端驗證表單功能。

```
<div class="container d-flex justify-content-center align-items-center" style="min-height: 100vh;">
  <div class="p-4 border rounded" style="width: 100%; max-width: 400px;">
    <form id="loginForm" action="login/login" method="post">
      <div class="mb-3">
        <label for="loginId" class="form-label">使用者名稱：</label>
        <input type="text" class="form-control" id="loginId" name="loginId" required>
        <div id="emailHelp" class="form-text"><c:out value="${loginId}"/></div>
      </div>
      <div class="mb-3">
        <label for="password" class="form-label">密碼：</label>
        <input type="password" class="form-control" id="password" name="password" required>
      </div>
      <div class="form-check form-check-inline">
        <input class="form-check-input" type="radio" name="userType" id="user" value="1">
        <label class="form-check-label" for="user">一般用戶</label>
    </div>
    <div class="form-check form-check-inline">
        <input class="form-check-input" type="radio" name="userType" id="admin" value="2">
        <label class="form-check-label" for="admin">管理者</label>
    </div>
      <button type="submit" class="btn btn-primary">提交</button> 
      <a class="btn btn-success" href="register/register" style="text-decoration: none;">註冊</a>
    </form>
  </div>
</div>
```

```
<div class="container d-flex justify-content-center align-items-center"
style="min-height: 100vh;">
```

這樣寫來維持子元素來水平、垂直置中。

```
<form id="loginForm" action="<%=request.getContextPath()%>/login/login"
method="post">
```

這種 url 路徑，可直接對應 Struts 2 的 url mapping。

傳統 JavaScript 表單提交、前後端驗證表單功能：

```
<script type="text/javascript">
    function validateAndSubmit() {
        let loginId = $("input[name='loginId']").val().trim();
        let password = $("input[name='password']").val().trim();

        if (!loginId) {
            alert("使用者名稱不能為空");
            return;
        }

        if (!password) {
            alert("密碼不能為空");
            return;
        }

        $("#loginForm").submit();
    }
</script>
```

5.2 提交到後端：

5.2.1 設置 url 與 Struts 2 Action 的具體方法，之間綁定的方式：

struts-login.xml：

```
<struts>
    <package name="login" extends="default" namespace="/login">
        <interceptors>

            覆寫父 package（繼承自 "default"）的預設攔截器 default-interceptor-ref，
        <default-interceptor-ref name="loginStack" />

            定義登入的 Action：
        <action name="login" class="com.example.action.LoginAction" method="doLogin">
            <!--
                若在這裡手動加入 interceptor-ref，則會覆蓋 default-interceptor-ref，
                這樣其他攔截器（如 fileUpload）將無法生效，需謹慎使用。
            -->
            <!-- <interceptor-ref name="authenticationInterceptor" /> -->

                當驗證未通過（輸入錯誤）時，重新導向至登入頁面
            <result name="input" type="redirect">/login.jsp</result>

            <!--
                當登入成功時，導向至首頁
            -->
            <result name="success" type="redirect">/home/index</result>
        </action>
```

struts.xml 的 package tag 作用是對專案功能做出分類；

struts.xml 的 action tag 作用是功能內部行為做出分類。

5.2.2 提交給 Struts 2 Action，將登入資訊寫入 VO、session：

loginAction：

```
private User user;

public User getModel() {
        if (user == null) {
            user = new User();
        }
        return user;
    }
```

在 Struts 2，如果要將前端表單屬性值，在 Action 綁定到某個 VO，就必須於該 action，宣告 VO+定義 getModel。

定義 getModel 之後，表單屬性值會在 Struts filter 攔截到表單之時，先呼叫對應 action 的 getModel，取得 VO，再將表單資料寫入 VO。

表單屬性，可以不用 struts tag 來表示。

```
public String doLogin() {
    // 檢查是否提供了登入 ID 和密碼
    if (user == null || user.getLoginId() == null || user.getPassword() == null) {
        getSession().setAttribute("msg", "請提供有效的帳號與密碼");
        return INPUT;
    }

    // 呼叫 UserService 來獲取使用者資料
    user = userService.getLoginUser(user);

    // 如果用戶資料存在且登入 ID 正確
    if (user != null && !"".equals(user.getLoginId())) {
        // 登入成功，將用戶資料放入 session 中
        getSession().setAttribute(ConstantName.SESSION_USER, user);
        return SUCCESS;
    }

    // 登入失敗，顯示錯誤訊息並返回輸入頁
    getSession().setAttribute("msg", "帳號或密碼錯誤");
    return INPUT;
}
```

5.2.3 登入過程中，輸入的密碼，如何與已經經過雜湊+加鹽的密碼來做比較：

```
bCryptPasswordEncoder.matches(request.getParameter("password"),
user.getPassword())
```

5.2.4 forward 到登入成功後，要進入的頁面：

struts-login.xml：

```
<result name="success" type="redirect">/home/index</result>
```

struts-home.xml：

```
<struts>
    <package name="home" extends="default" namespace="/home">
            這個 package（封裝作用域）名稱為 "home"，對應 /home 命名空間，

            index（首頁）Action：
        <action name="index" class="com.example.action.HomeAction" method="goHome">
            <result name="success">/WEB-INF/views/index.jsp</result>
        </action>
```

這樣就進入 EShop 首頁了。

那麼，為何要先寫

```
<result name="success" type="redirect">/home/index</result>
```

而不是直接 forward 到 index.jsp？

這是因為，要讓瀏覽器顯示的網址，能夠是自己要的，這樣就必須先用 redirect 而非 forward，顯示的網址變成自己要的之後，再進行 forward。

這個模式叫做 Post-Redirect-Get 模式，普遍用於需要切換瀏覽器 url，但對應的 jsp 檔又位於 WEB-INF 時，常用模式。

6 EShop 首頁功能：

6.1 首頁頁面概說：





這是用 html5 標準的導覽列(navigator)+main+footer 所建構的頁面、bootstrap 風格導覽列+透過 EL 呈顯 session 內的用戶資訊+透過 EL 與於 Struts 2 action 綁定的用戶屬性，決定不同用戶的權限+使用 jsp 語法與 struts 2 url mapping 來設置超連結的 url。

index.jsp：



```
struts-home.xml | index.jsp × | struts-logi... | OrderReposi... | OrderAction... | struts-order... | login.jsp | LoginAction...
12 <body class="d-flex flex-column min-vh-100">
13 <nav class="navbar navbar-expand bg-info">
14   <div class="container-fluid d-flex justify-content-between align-items-center">
15     <a class="navbar-brand d-flex justify-content-center align-items-center mx-3 my-auto" href="#">EShop</a>
16     <div class="d-flex justify-content-center align-items-center">
17         <ul class="navbar-nav d-flex justify-content-center align-items-center">
18           <li class="nav-item dropdown">
19             <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-toggle="dropdown" aria-
20               購買商品
21             </a>
22             <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
23               <li><a class="dropdown-item" href="<%= request.getContextPath() %>/product/productList">商品列表</a></li>
24               <c:if test="${session_user.userType eq 2}">
25                 <li><a class="dropdown-item" href="#">商品管理</a></li>
26               </c:if>
27             </ul>
28           </li>
29
30           <c:if test="${session_user.userType eq 1}">
31               <li class="nav-item">
32                 <a class="nav-link" href="<%= request.getContextPath() %>/cart/queryCart?loginId=${session_user.loginId}">
33               </li>
34           </c:if>
35
36           <li class="nav-item dropdown">
37             <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-toggle="dropdown" aria-
38               訂單資訊
39             </a>
40             <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
41               <c:if test="${session_user.userType eq 1}">
42                   <li><a class="dropdown-item" href="<%= request.getContextPath() %>/order/queryOrderList">訂單詳細資訊</
43               </c:if>
44               <c:if test="${session_user.userType eq 2}">
45                 <li><a class="dropdown-item" href="<%= request.getContextPath() %>/order/queryAllOrder">訂單詳細資訊</a></l
46                 <li><a class="dropdown-item" href="#">訂單狀態更新</a></li>
47               </c:if>
48             </ul>
49           </li>

50           <li class="nav-item dropdown">
51             <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-toggle="dropdown" aria-
52               使用者管理
53             </a>
54             <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
55               <li><a class="dropdown-item" href="#">密碼重置</a></li>
56               <li><a class="dropdown-item" href="#">修改個人資料</a></li>
57               <li><a class="dropdown-item" href="#">管理收貨地址</a></li>
58             </ul>
59           </li>
60           <li class="nav-item">
61             <a class="nav-link" href="<%= request.getContextPath() %>/home/logout">登出</a>
62           </li>
63           </ul>
64           你好！${session_user.loginId}！
65           <form class="d-flex justify-content-center align-items-center my-0">
66               <input class="form-control mx-1" type="search" placeholder="Search" aria-label="Search">
67               <button class="btn btn-outline-success mx-1" type="submit">Search</button>
68           </form>
69       </div>
70   </div>
71 </nav>
72 <main class="flex-grow-1">
73
74 </main>
75 <footer class="d-flex justify-content-center align-items-center bg-info text-dark py-3">eshop</footer>
76 </body>
```

6.2 前端頁面：

首頁頁面以 bootstrap 作為排版框架：

首先，body 部分設置為 flexbox，以方便對子元素排版，並設定該 body 必須是剛好佔有整個瀏覽器頁面的：

```
<body class="d-flex flex-column min-vh-100">
```

接著，導覽列部分，先直接套用 bootstrap 官網上的範例，再進行客製化修改：

```
<div class="container-fluid
```

```
        d-flex justify-content-between align-items-center px-3">
```

這裡一樣設置為 flexbox，以方便對子元素排版。

```
<a class="navbar-brand d-inline-flex justify-content-center align-items-
center" href="#">EShop</a>
```

這裡一樣設置為 flexbox，以方便對內部文字水平、垂直置中。

```
<div class="d-flex justify-content-center align-items-center">
        <ul class="navbar-nav
        d-flex justify-content-center align-items-center">
```

這裡一樣設置為 flexbox，以方便對子元素排版。

```
<form class="d-flex justify-content-center align-items-center">
            <input class="form-control mx-1" type="search"
placeholder="Search" aria-label="Search">
            <button class="btn btn-outline-success mx-1"
type="submit">Search</button>
        </form>
```

上面之所以用 mx-1，而非使用 px-1 來隔開彼此，是因為 px 是作用於 border 內部，但這裡是要在 border 外部來隔開，且 mx 在 flexbox 中，不會產生元素之間的 mx，互相影響干擾的問題。

6.3 後端渲染資料：

使用 jsp 語法與 struts 2 url mapping 來設置超連結的 url：23、32、42、46、61 行。

透過 EL 與於 Struts 2 action 綁定的用戶屬性，決定不同用戶的權限：24、30、41、44 行。

透過 EL 呈顯 session 內的用戶資訊：64 行。

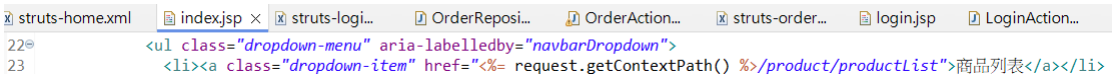
7 商品 CRUD 功能：

7.1 商品查詢功能：

7.1.1 於 index.jsp 點擊連結：



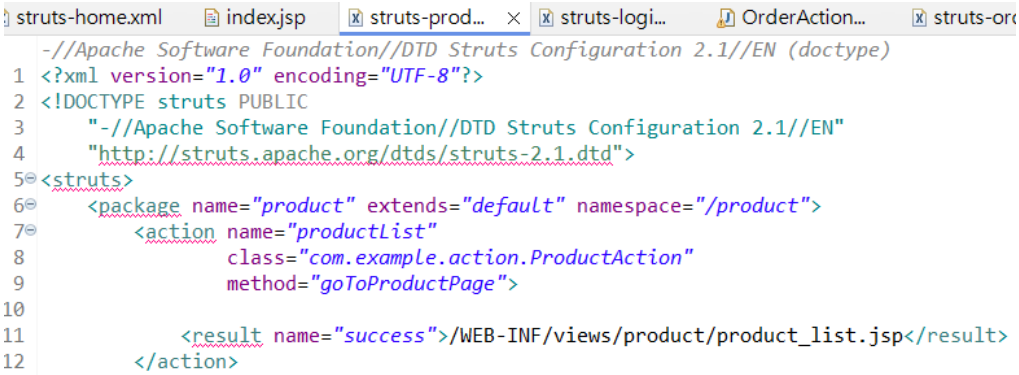
對應到以下超連結：



接著會進入 struts 2 url mapping->action->service->dao->資料庫，再從資料庫一路走回來的流程：

struts 2 url mapping：

檔名：struts-product.xml



struts 2 url mapping 的具體寫法：6、7 行。

7.1.2 action：

檔名：ProductAction：

```
11 public class ProductAction extends BaseAction {
12
13     @Autowired
14     private ProductService productService;
15
16     private List<Object[]> productList;
17     private Object[] productDetail;
18
19      public String goToProductPage() throws Exception {
20         //HttpSession session= getSession();
21         //session.setAttribute("productList", productSer
22         productList = productService.queryAllProduct();
23         return SUCCESS;
24     }
```

第 13、14 行：設定這裡要依賴注射 ProductService 物件。

第 22 行：呼叫方法來查詢商品，故呼叫 ProductService 物件之方法，再進一步呼叫 ProductDao 物件之方法。

回傳到 action 的 productList 後，透過 32-34 行的語法：

```
32     public List<Object[]> getProductList() {
33         return productList;
34     }
```

即可直接於 jsp 檔，透過 struts tag + productList 名稱：

```
<s:iterator value="productList" var="product">
```

這段可以透過 getProductList，取值出來。這就是 Struts 2 的自動綁定功能。

第 23 行：執行 forward，到 product_list.jsp

7.1.3 ProductService：

```
12 @Service
13 public class ProductServiceImpl implements ProductService{
14
15     @Autowired
16     private ProductDao productDao;
17     @Override
18     public List<Object[]> queryAllProduct() {
19
20         return productDao.queryAllProduct();
21     }
```

第 12 行：註冊該 bean 到 Spring 容器。

第 15、16 行：設定這裡要依賴注射 ProductDao 物件。

7.1.4 ProductDaoImpl：

```
12 @Repository
13 public class ProductDaoImpl implements ProductDao{
14     @Autowired
15     SessionFactory sessionFactory;
16
17     @Override
18     public List<Object[]> queryAllProduct() {
19         String hql = "select p.prodId,p.prodName,p.prodPrice FROM Product p";
20         List<Object[]> products = sessionFactory.getCurrentSession().createQuery(hql)
21                                   .getResultList();
22         return products;
23     }
```

第 12 行：註冊該 bean 到 Spring 容器。

第 14、15 行：設定這裡要依賴注射 Hibernate 的 sessionFactory 物件。

第 19 行：以 Product 這個 Entity 為基礎，設置查詢商品要用的 HQL。

第 20、21 行：以 Hibernate 傳統方式執行查詢。

7.1.5 forward 到 product_list.jsp：

```
struts-home.xml | ProductServi... | index.jsp | product_lis... | ProductActi... | struts-prod... | struts-logi... | LoginAction...
20     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
21 </head>
22 <body>
23     <%@ include file="../exampleNav.jspf" %>
24     <h2 class="my-2">商品列表</h2>
25
26     <div class="w-75 mx-auto">
27         <table id="productTable" class="table table-striped table-bordered">
28             <thead class="table-primary">
29             <tr>
30                 <th>編號</th>
31                 <th>商品名稱</th>
32                 <th>價格</th>
33                 <th style="width: 10%;"></th>
34             </tr>
35             </thead>
36             <tbody>
37             <s:iterator value="productList" var="product">
38                 <tr>
39                     <td><s:property value="#product[0]"/></td>
40                     <td><s:property value="#product[1]"/></td>
41                     <td><s:property value="#product[2]"/></td>
42                     <td class="d-flex justify-content-center align-items-center">
43                         <a href="<%= request.getContextPath() %>/product/productDetail?prodId=<s:property value="#product[0]"/>"
44                         class="btn btn-primary" role="button">購買商品</a>
45                     </td>
46                 </tr>
47             </s:iterator>
48             </tbody>
49         </table>
50     </div>
51
52     <script>
53         $(document).ready(function () {
54             $('#productTable').DataTable({
55                 language: {
56                     url: "//cdn.datatables.net/plug-ins/1.13.6/i18n/zh-HANT.json"
57                 }
58             });
59         });
60     </script>
61 </body>
```

第 37-47 行，使用 struts tag，將存於後端 action 屬性的商品物件，一一取出呈顯。

這裡的 Struts tag，可以用 JSTL 來取代嗎？可以，但功能會因此受限。

JSTL：<c:forEach var="product" items="${productList}">

Struts tag：<s:iterator value="productList" var="product">

為何兩者的 var，位置是相反的？

因為前者試圖盡可能與 Java 語法匹配；後者則是試圖表示先有資料再有變數去接收。

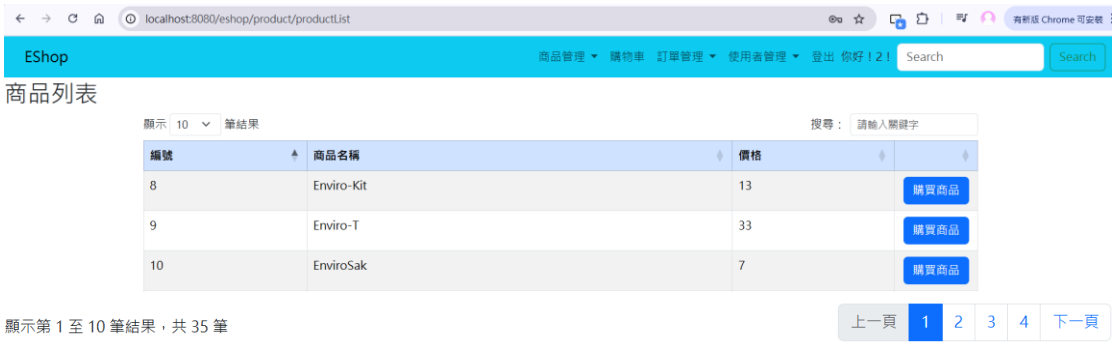
可以固定讓 var 在前面嗎？

不行，兩者寫法都是固定的。

第 43-44 行，利用商品 id 這個值，設置購買該商品的按鈕，以做到新增商品至購物車之功能。

第 52-60 行，透過導入 JQuery dataTable，優化美化查詢結果、並額外針對查詢結果，增加查詢、結果排序、分頁等功能。

7.1.6 查詢畫面：



8 購物車 CRUD 功能：

8.1 新增商品至購物車之功能：

8.1.1 於 product_list.jsp，新增商品之按鈕：

```
<td class="d-flex justify-content-center align-items-center">
    <a href="<%= request.getContextPath() %>/product/productDetail?prodId=<s:property value="#product[0]"/>"
    class="btn btn-primary" role="button">購買商品</a>
</td>
```

8.1.2 struts-product.xml，Struts 2 url mapping：

```
<struts>
    <package name="product" extends="default" namespace="/product">
        <action name="productList"
                class="com.example.action.ProductAction"
                method="goToProductPage">

            <result name="success">/WEB-INF/views/product/product_list.jsp</result>
        </action>
        <action name="productDetail"
                class="com.example.action.ProductAction"
                method="queryProductDetail">

            <result name="success">/WEB-INF/views/product/product_buy.jsp</result>
        </action>
    </package>
</struts>
```

8.1.3 依序進入 action、service、dao：

```
    public String queryProductDetail() {
        String prodId = getRequest().getParameter("prodId");
        productDetail = productService.queryProductById(Integer.parseInt(prodId));
        return SUCCESS;
    }
public Object[] queryProductById(Integer id) {
    return productDao.queryProductById(id);
}
```

```
public Object[] queryProductById(Integer id) {
    String hql = "select p.prodType.prodType,p.prodLine.prodLine,p.prodName,p.prodPrice,p.prodId FROM Product p where p.prodId=:id";
    Object[] product = (Object[])sessionFactory.getCurrentSession().createQuery(hql)
                    .setParameter("id", id).getResultList().get(0);
    return product;
}
```

action 定義了：

```
private Object[] productDetail;
```

只要將查詢到的資料塞入該物件，即可於 JSP 頁面，透過 Struts 2 tag，從 Struts 2 的 valuestack(不是從 request 取出)，取查詢值出來呈顯。

8.1.4 商品明細頁面：

product_buy.jsp：

```
<div class="d-flex justify-content-center align-items-center">
      <!-- 商品圖片 -->
      <img src="<%= request.getContextPath()
%>/product/productImage?prodId=<s:property value='productDetail[4]' />"
            class="img-fluid rounded" alt="商品圖片">
      <!-- 這個 src，是一被瀏覽器解析到，就立刻執行 -->
      <div style="padding:0 20px;"></div>
      <!-- 商品資訊 -->
      <div>
        <h2 class="fw-bold"><s:property value="productDetail[0]" /></h2>
        <h3 class="mb-4"><s:property value="productDetail[1]" /></h3>

        <h3 class="mb-4"><s:property value="productDetail[2]" /></h3>
        <h3 class="mb-4">$<span class="text-danger"><s:property
value="productDetail[3]" /></span>元</h3>


         <c:set var="amount1" value="<s:property value='productDetail[5]'
/>"/>
         <s:set var="amount1" scope="request" value="<s:property
value='productDetail[5]' />"/>
         <s:set var="amount" scope="request" value="%{productDetail[5]}"/>
         已選取
         <select class="form-select d-inline-block w-auto" id="quantity">
             <c:choose>
                 <c:when test="${amount==0}">
                     <option disabled>0</option>
                 </c:when>
                 <c:otherwise>
                     <c:forEach var="i" begin="1" end="${amount}">
                         <c:if test="${i==1}">
                             <option value="${i}" selected>${i}</option>
                         </c:if>
                         <c:if test="${i!=1}">
                             <option value="${i}">${i}</option>
```

```
                        </c:if>
                    </c:forEach>
                </c:otherwise>
            </c:choose>
        </select>
        件商品

        <p class="mb-4">目前還剩<s:property value="productDetail[5]" />個商品</p>
        <div class="d-grid gap-2 d-md-block">
        <c:choose>
            <c:when test="${amount==0}">
            <a id="addToCart" class="btn btn-primary me-2 disabled" aria-disabled="true">加入購物車</a>
        </c:when>
            <c:otherwise>
                <a id="addToCart" class="btn btn-primary me-2">加入購物車</a>
            </c:otherwise>
        </c:choose>
        <a id="collect" class="btn btn-secondary me-2">收藏</a>
        </div>
      </div>
    </div>
```

`<s:property value="productDetail[0]" />`

透過 Struts 2 tag 取查詢值出來呈顯。

8.1.4.1 呈顯圖片：

```
<img src="<%= request.getContextPath() %>/product/productImage?prodId=<s:property value='productDetail[4]' />" class="img-fluid rounded" alt="商品圖片">
```

這個 src，是一被瀏覽器解析到，就立刻執行。

8.1.4.2 呈顯剩餘可選商品數量，且設置下拉式選單讓用戶可選擇數量：

```
<s:set var="amount" scope="request" value="%{productDetail[5]}"/>
```

已選取

```
<select class="form-select d-inline-block w-auto" id="quantity">
    <c:choose>
        <c:when test="${amount==0}">
            <option disabled>0</option>
        </c:when>
        <c:otherwise>
            <c:forEach var="i" begin="1" end="${amount}">
                <c:if test="${i==1}">
                    <option value="${i}" selected>${i}</option>
                </c:if>
                <c:if test="${i!=1}">
                    <option value="${i}">${i}</option>
                </c:if>
            </c:forEach>
        </c:otherwise>
    </c:choose>
</select>
```

件商品

```
<s:set var="amount" scope="request" value="%{productDetail[5]}"/>
```

將查詢到的商品數量 assign 給 amount，再把 amount 放進 request scope，方便 JSP EL/JSTL 直接以${amount}方式來使用。

可以寫成

```
<c:set var="amount1" value="<s:property value='productDetail[5]' />"/>
```

？不行，因為 JSTL 會先試圖讀取 value，但這時 Struts tag 尚未被解析。

那可以寫成

```
<s:set var="amount1" scope="request" value="<s:property
value='productDetail[5]' />"/>
```

？不行，因為 Struts tag 內不能再包其他 Struts tag。

```
<c:forEach var="i" begin="1" end="${amount}">
```

var="i"，然後要使用這個值時，為何要用 EL？

因為 i 是 JSTL 變數，這種變數可用 EL 取出。

8.1.4.3 如果商品數量為 0，則將購買鈕設為 disable：

```
<c:choose>
    <c:when test="${amount==0}">
        <a id="addToCart" class="btn btn-primary me-2 disabled" aria-
disabled="true">加入購物車</a>
    </c:when>
    <c:otherwise>
        <a id="addToCart" class="btn btn-primary me-2">加入購物車</a>
    </c:otherwise>
</c:choose>
```

8.1.4.4 實際頁面：



8.1.5 將商品加入購物車：

接著，以 JavaScript 原生的 fetch API，做到將商品置於購物車，且能回傳成功訊息到同一頁面，並彈出 bootstrap 風格視窗來告知用戶結果的效果：

```
function addToCart() {
        fetch('<%= request.getContextPath() %>/cart/addToCart', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
                 'X-Requested-With': 'XMLHttpRequest'
```

```
            },
            body: "prodId=<s:property value='productDetail[4]'
/>&userId=${session_user.loginId}&quantity="+document.getElementById('quant
ity').value
})
.then((res)=>{return res.json();})
                    .then((data) => {
                          if(data.message=='商品成功加入購物車！'){
                                var myModal = new
bootstrap.Modal(document.querySelector('#modal1'));
                                myModal.show();
                          }
                    })
                    .catch(() => {
                      alert('加入購物車失敗，請稍後再試。');
                    });
                  }
```

`'Content-Type': 'application/x-www-form-urlencoded'`

`{ 'ContentType': 'application/x-www-form-urlencoded' }`
跟`{ ContentType: 'application/x-www-form-urlencoded' }`
key 都會被瀏覽器被解析成字串值 ContentType，所以兩者才會一樣。

`headers:`

headers 屬性，內容會全被寫入 request header
為何要叫做「headers」屬性，不直接叫 header 就好？上面一個 key-value 對就是一個 header，這裡有兩個，故就是 headers。

`(res)=>{return res.json();}`

res 是 javascript 的 httpResponse 型別物件；
res.json()是 httpresponse body(這為 json 格式，因 Struts 2 設定檔有相關設定)，轉成普通 js 物件；
return 則是在 promise 物件狀態改為 fulfilled 時，再將該 js 物件 return 出來。

```
(data) => {
            if(data.message=='商品成功加入購物車！'){
                  var myModal = new
bootstrap.Modal(document.querySelector('#modal1'));
                  myModal.show();
                  }
```

```
        }
```

這個 data 是 js 物件

在 action 加入的屬性名-值對，就在該物件內，可直接用屬性名將值取出。

呼叫該 API：

```
$(document).ready(function(){
    document.getElementById("addToCart").addEventListener("click",function(){
        addToCart();
    });
```

以 url 進行 Struts 2 url mapping：

struts-cart.xml：

```
-//Apache Software Foundation//DTD Struts Configuration 2.1//EN (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
    "http://struts.apache.org/dtds/struts-2.1.dtd">
<struts>
    <package name="cart" extends="json-default" namespace="/cart">
        <action name="addToCart"
                class="com.example.action.CartAction"
                method="addToCart">

            <result name="success" type="json"/>
        </action>
```

進入對應 action 方法：

```
public class CartAction extends BaseAction{

    @Autowired
    private CartService cartservice;

    private String message;

    private List<Object[]> cartList;

    public String addToCart() {
        HttpServletRequest request = getRequest();
        String userId = request.getParameter("userId");
        String prodId = request.getParameter("prodId");
        //String quantity = request.getParameter("quantity");

        int result = cartservice.addToCart(Integer.parseInt(userId),Integer.parseInt(prodId));

        if(result>0) {
            message="商品成功加入購物車！";
        }

        return SUCCESS;
    }
```

回到剛剛的 struts-cart.xml：

```
ProductServi...  struts-cart.xml ×  product_lis...  ProductActi...  struts-prod...
   -//Apache Software Foundation//DTD Struts Configuration 2.1//EN (doctype)
 1 <?xml version="1.0" encoding="UTF-8"?>
 2 <!DOCTYPE struts PUBLIC
 3     "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
 4     "http://struts.apache.org/dtds/struts-2.1.dtd">
 5 <struts>
 6     <package name="cart" extends="json-default" namespace="/cart">
 7         <action name="addToCart"
 8                 class="com.example.action.CartAction"
 9                 method="addToCart">
10
11             <result name="success" type="json"/>
12         </action>
```

表示從後端回傳的成功訊息，要被解析成 json 格式物件回傳到前端。

前端回呼函數接受到該物件後，進行以下操作來取出要呈顯的字串：

```
function addToCart() {
      fetch('<%= request.getContextPath() %>/cart/addToCart', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/x-www-form-urlencoded'
        },
        body: "prodId=<s:property value='productDetail[4]' />&userId=${session_user.loginId}"
      })
      .then((res)=>{return res.json();})
      .then((data) => {
            if(data.message=='商品成功加入購物車！'){
                  var myModal = new bootstrap.Modal(document.querySelector('#modal1'));
                  myModal.show();
            }
      })
```

商品成功加入購物車後，要彈出的 bootstrap 風格視窗：

```
<div id="modal1" class="modal fade" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">加入購物車</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        <p>加入購物車成功！</p>
      </div>
      <div class="modal-footer">
        <a class="btn btn-secondary" data-bs-dismiss="modal">取消</a>
        <a href="<%= request.getContextPath() %>/cart/queryCart?loginId=${session_user.loginId}" class="btn btn-pr
      </div>
    </div>
  </div>
</div>
```

8.1.6 實際頁面：

以 bootstrap 搭配 Struts tag，來呈顯商品動態資料的簡易 html 架構：



彈出 bootstrap 風格視窗來告知用戶結果：



按下確定鈕，即可前往購物車頁面。

8.2 進入購物車頁面：

8.2.1 按下該按鈕後，根據按鈕的 href，進行 Struts 2 url mapping：

ProductServi... | struts-cart.xml | product_lis... | ProductActi... | struts-prod...

```
-//Apache Software Foundation//DTD Struts Configuration 2.1//EN (doctype)
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
4      "http://struts.apache.org/dtds/struts-2.1.dtd">
5  <struts>
6      <package name="cart" extends="json-default" namespace="/cart">
7          <action name="addToCart"
8                  class="com.example.action.CartAction"
9                  method="addToCart">
10
11             <result name="success" type="json"/>
12         </action>
13
14         <action name="queryCart"
15                 class="com.example.action.CartAction"
16                 method="queryCart">
17
18             <result name="success">/WEB-INF/views/cart/cart_list.jsp</result>
19         </action>
20     </package>
21 </struts>
```

8.2.2 將購物車表單與 Action 內的屬性綁定：

cart_list.jsp：

```
<form action="<%= request.getContextPath() %>/purchase/goToPurchase"
method="post" id="purchaseForm">
        <table id="myTable" class="table table-striped table-bordered">
          <thead class="table-primary">
            <tr>
              <th>商品名稱</th>
              <th>價格</th>
              <th>數量</th>
              <th>是否購買</th>
            </tr>
          </thead>
          <tbody>
```

```
            <% int index = 0; %>
              <s:iterator value="cartList" var="cart" status="st">
                  <tr>
                      <td><s:property value="#cart[0]" /></td>
                      <td><s:property value="#cart[1]" /></td>
                      <td>
                         <input type="hidden"
name="cartItem[<s:property value='#st.index'/>].prodId"
value="<s:property value='#cart[3]'/>"
id="cartItem_<s:property value='#st.index'/>_prodId">
                         <select class="form-select"
name="cartItem[<s:property value='#st.index'/>].quantity">
                           <s:iterator begin="1" end="%{#cart[2]}" var="i">
                              <option value="<s:property value='#i' />"
                                  <s:if test="#i == 1">selected</s:if>>
                                  <s:property value="#i" />
                              </option>
                           </s:iterator>
                         </select>
                      </td>
                      <td>
                          <div class="form-check p-0 m-0 d-flex justify-
content-center align-items-center">
                              <s:checkbox theme="simple" class="form-
check-input" name="cartItem[%{#st.index}].selected" fieldValue="true"/>
                          </div>
                      </td>
              </tr>
              <% index++; %>
              </s:iterator>
    </tbody>
</table>
</form>
```

```
<s:iterator value="cartList" var="cart" status="st">
<s:property value="#cart[0]" />
<s:property value="%{#cart[0]}" />
```

兩種寫法的差異？一樣。

為何要有#字號？因為這是 OGNL 的 Context Map 變數，不是放在 ValueStack

上。

```
<s:property value='#st.index'/>
```

跟`<s:property value='%{#st.index}'/>`

一樣？一樣。

```
<s:iterator value="cartList" var="cart" status="st">
<s:iterator begin="1" end="%{#cart[2]}" var="i">
```

跟`<s:iterator begin="1" end="#cart[2]" var="i">`

一樣？不一樣，因為 value 會把值直接當 Struts 2 值解析，end 只會當成字串來解析，需要加上%{}才能強制當成 Struts 2 值。

```
end="%{#cart[2]}"
```

不可寫成

```
end="<s:property value='#cart[2]' />"
```

因為 tag 的值不能被嵌套進另一個 struts tag。

```
<s:checkbox theme="simple" class="form-check-input"
name="cartItem[%{#st.index}].selected" fieldValue="true"/>
```

這樣才可讓該 checkbox 不會有額外樣式。

表單本身做 getModel 較為方便，故最好也能將其表示成物件，例如 CartForm：

```
public class CartForm {
    private List<CartItem> cartItem;
```

在 Action 就透過 getModel 取出該物件：

```
private CartForm model = new CartForm();

    public CartForm getModel() {
        return model;
    }
public String goToPurchase() {

        List<CartItem> cartitemList = model.getCartItem();
```

`List<CartItem> cartitemList = model.getCartItem();`

直接從表單對應物件取出表單內的資料。

為何表單的對應物件不用在 jsp 檔表示出來？當 Action 實作，用了 ModelDriven，CartForm 會自動成為根物件，不必在 JSP 額外指名，直接從 CartForm 的屬性開始寫

這裡的 ModelDriven 是類別加上攔截器

8.2.3 實際頁面：



這是該用戶，購物車內所有的商品；可以看到選單最右側，有 checkbox 可以選擇真正要購買的商品。
用 checkbox 選擇要購買的商品後，即可點擊購買鈕進行購買，接著即進入付款頁面。

9 付款/訂單 CRUD 功能：

9.1 checkPurchase.jsp：

畫面是以 bootstrap 風格表單為主的 html 架構；目前只有模擬信用卡付款的功能(下圖紅框處)：

```
            const form = document.getElementById('creditCardForm');

            if (creditRadio.checked) {
              form.style.display = 'block';
            } else {
              form.style.display = 'none';
            }
        }
    </script>
</head>
<body>
<%@ include file="../exampleNav.jspf" %>
    <h2 class="my-2">購買商品</h2>
    <form id="paymentForm" action="<%= request.getContextPath() %>/purchase/purchase" method="post">


        <div class="container mt-5" style="max-width: 600px;">
            <h4 class="mb-4">選擇付款方式</h4>
              <!-- 付款方式選擇 -->
              <div class="mb-3">
                <div class="form-check">
                  <input class="form-check-input" type="radio" name="paymentMethod" id="creditCardOption" value="credit" onclick="toggleCreditCardForm()"
                  <label class="form-check-label" for="creditCardOption">
                    信用卡付款
                  </label>
                </div>
                <div class="form-check">
                  <input class="form-check-input" type="radio" name="paymentMethod" id="atmOption" value="atm" onclick="toggleCreditCardForm()">
                  <label class="form-check-label" for="atmOption">
                    ATM 轉帳
                  </label>
                </div>
              </div>

              <!-- 送出 -->
              <div class="d-grid gap-2 col-6 mx-auto mt-2">
                <button type="submit" class="btn btn-primary">確認付款</button>
                <a href="<%= request.getContextPath() %>/cart/queryCart?loginId=${session_user.loginId}" class="btn btn-secondary" >取消</a>
              </div>
          </div>
      </form>
</body>
</html>
```

實際畫面：



按下確認付款按鈕後，即可進入綠界模擬付款頁面。

9.2 表單 action 對 Struts 2 url mapping：

struts-purchase.xml：

```
-//Apache Software Foundation//DTD Struts Configuration 2.1//EN (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE struts PUBLIC
    "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
    "http://struts.apache.org/dtds/struts-2.1.dtd">
<struts>
    <package name="purchase" extends="json-default" namespace="/purchase">
        <action name="purchase"
                class="com.example.action.PurchaseAction"
                method="purchase">
        </action>
```

9.3 進入對應 action(PurchaseAction)之方法(purchase)：

內含綠界 SDK API，可直接模擬信用卡付款流程：

struts-purc... | PurchaseActi... | checkPurchas... | struts-cart.xml | product_lis... | ProductActi... | struts-prod... | product_buy.jsp

```
51
52    public void purchase() throws IOException {
53
54        User user = (User)getSession().getAttribute(ConstantName.SESSION_USER);
55
56        String orderNum = orderService.createOrder(user.getLoginId(), (List<CartItem>)getSession().getAttribute("cartItem"));
57
58        getSession().setAttribute("orderNum", orderNum);
59
60        AioCheckOutALL obj = new AioCheckOutALL();
61        String timeStamp = new SimpleDateFormat("yyyyMMddHHmmss").format(new Date());
62
63        obj.setMerchantTradeNo("EC" + timeStamp); // 訂單編號
64        obj.setMerchantTradeDate(new SimpleDateFormat("yyyy/MM/dd HH:mm:ss").format(new Date()));
65        obj.setTotalAmount("100");
66        obj.setTradeDesc("綠界 SDK 測試單");
67        obj.setItemName("測試商品x1");
68        obj.setReturnURL("http://localhost:8080/eshop/purchase/cashFinish");
69        obj.setClientBackURL("http://localhost:8080/eshop/purchase/goToCashSuccess");
70        obj.setNeedExtraPaidInfo("N");
71        //obj.setChoosePayment("Credit");
72
73        // 產生 HTML 表單（含自動送出）
74        String form = all.aioCheckOut(obj, null);
75
76        // 寫到 response，送出給綠界
77        HttpServletResponse response = ServletActionContext.getResponse();
78        response.setContentType("text/html;charset=UTF-8");
79        PrintWriter out = response.getWriter();
80        out.print(form);
81        out.flush();
82
83        return ; // 直接寫出 HTML，不走 view 路徑
84    }
```

必須注意的是下圖紅框處：

```
63        obj.setMerchantTradeNo("EC" + timeStamp); // 訂單編號
64        obj.setMerchantTradeDate(new SimpleDateFormat("yyyy/MM/dd HH:mm:ss").format(new Date()));
65        obj.setTotalAmount("100");
66        obj.setTradeDesc("綠界 SDK 測試單");
67        obj.setItemName("測試商品x1");
68        obj.setReturnURL("http://localhost:8080/eshop/purchase/cashFinish");
69        obj.setClientBackURL("http://localhost:8080/eshop/purchase/goToCashSuccess");
70        obj.setNeedExtraPaidInfo("N");
71        //obj.setChoosePayment("Credit");
```

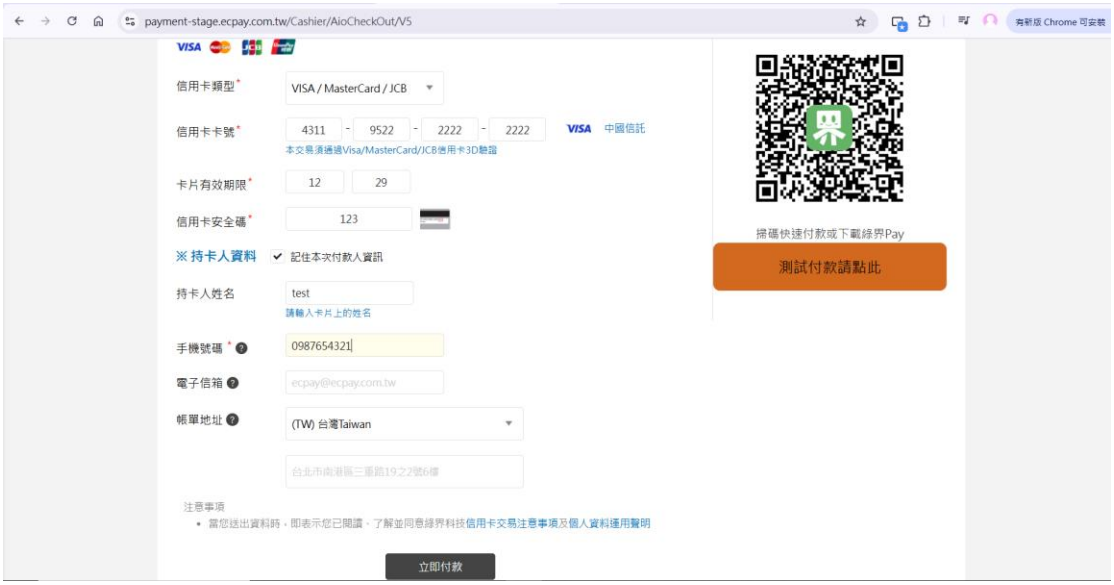
第 68 行：設置付款流程完成後，要到的資源所對應的 url。

第 69 行：設置付款流程進行中，要到的資源所對應的 url(通常顯示「付款完成，請稍待.....」這樣的頁面)。

經過這段綠界的測試用程式碼後，即可抵達綠界測試用付款頁面：



可輸入測試用信用卡資料：



按下「立即付款」按鈕，即可執行模擬信用卡付款流程，付款成功後，就要進入「付款完成，要返回專案頁面之前」，中間的銜接頁面。

9.4 付款完成，即將跳回專案頁面之前的銜接頁面(可以在這裡，設定付款完成後要做的動作)：

goToCashSuccess.jsp：

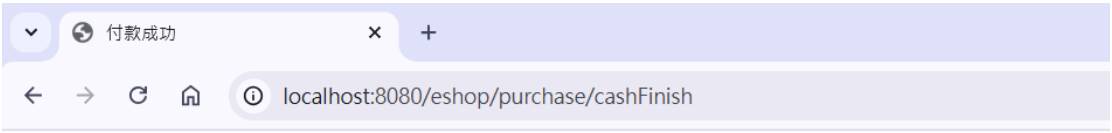
下圖即是該 jsp 檔之內容，紅框處表示當該銜接頁面，onload 完成即跳轉至 /purchase/cashFinish 這個 url，對應的資源(於這裡，具體設定付款完成後要做

的動作)。

struts-purc... goToCashSuc... PurchaseActi... checkPurchas... struts-cart.x

```
1  <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2  <!DOCTYPE html>
3  <html>
4  <head>
5    <meta http-equiv="refresh" content="0;url=/eshop/purchase/cashFinish" />
6    <title>付款成功導回中...</title>
7  </head>
8  <body>
9    <p>付款完成，正在為您導回...</p>
10 </body>
11 </html>
```

9.5 實際頁面：



付款完成，正在為您導回...

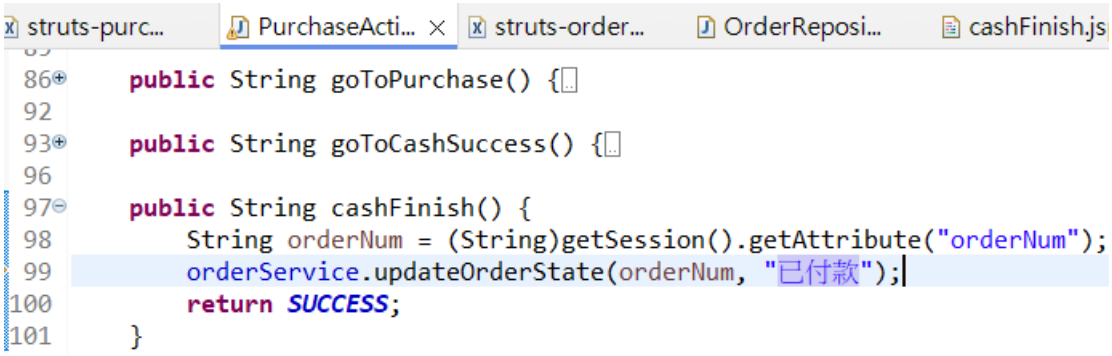
9.6 付款完成後動作：

這時，會進入/purchase/cashFinish，這個 url，對應的資源，這在 struts-purchase.xml 有設定具體要對應的資源(下圖紅框處)：



9.6.1 將訂單狀態改為「已付款」:

進入該 Action(PurchaseAction)的對應方法，設定當前訂單與「已付款」狀態之間的綁定：



接著經過 service、dao，於 dao，透過 Spring Data JPA 語法，修改訂單狀態為「已付款」:

透過 Spring Data JPA，定義的 OrderRepository，下圖紅框處即為修改訂單狀態之方法。

```java
package com.example.dao;

import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Integer> {

    void updateOrderState(@Param("ordNum") String ordNum, @Param("state") String state);

    List<Order> findByCustomer_CustId(Integer custId);

    @Modifying
    @Transactional
    @Query("UPDATE Order o SET o.state = :state WHERE o.ordNum = :ordNum")
    int updateOrderStateByOrdNum(@Param("state") String state, @Param("ordNum") String ordNum);
}
```

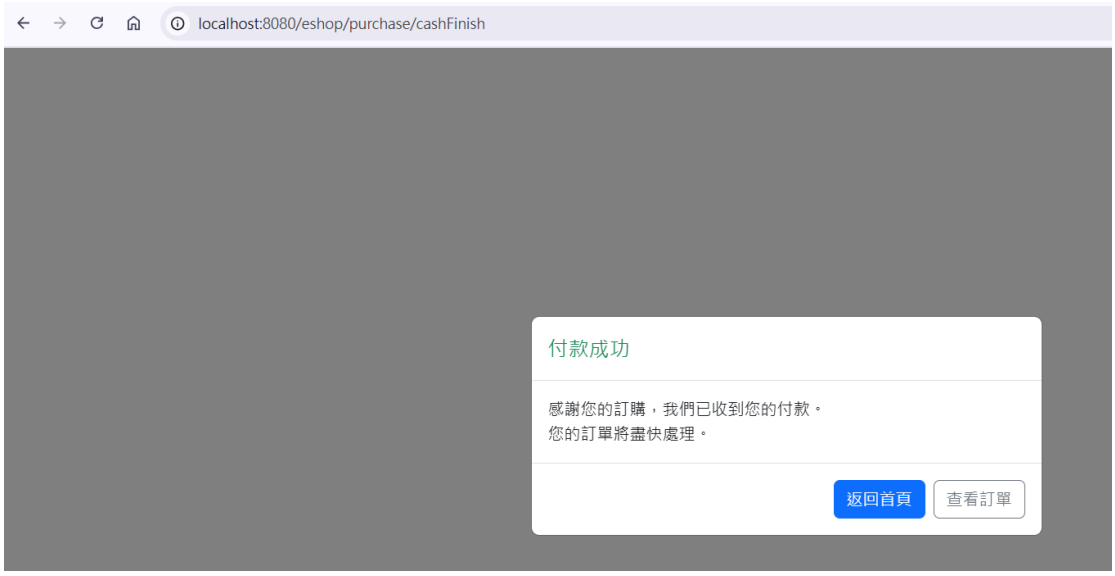
9.6.2 呈顯付款完成時，真正要呈顯的 jsp 頁面：

cashFinish.jsp：

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>付款成功</title>
    <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body>
    <!-- Bootstrap Modal 結構 -->
    <div class="modal fade" id="successModal" tabindex="-1" aria-hidden="true">
      <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">
          <div class="modal-header">
            <h5 class="modal-title text-success">付款成功</h5>
          </div>
          <div class="modal-body">
            感謝您的訂購，我們已收到您的付款。<br>
            您的訂單將盡快處理。
          </div>
          <div class="modal-footer">
            <a href="/eshop/home/index" class="btn btn-primary">返回首頁</a>
            <a href="/eshop/order/queryOrderList" class="btn btn-outline-secondary">查看訂單</a>
          </div>
        </div>
      </div>
    </div>
    <script>
      window.onload = function () {
        var modal = new bootstrap.Modal(document.getElementById('successModal'));
        modal.show();
      };
    </script>
</body>
</html>
```

可以看出，一旦成功進入該頁面，即會執行第 32 行，就會彈出 bootstrap 風格視窗，表示付款成功完成。

9.7 付款成功頁面：

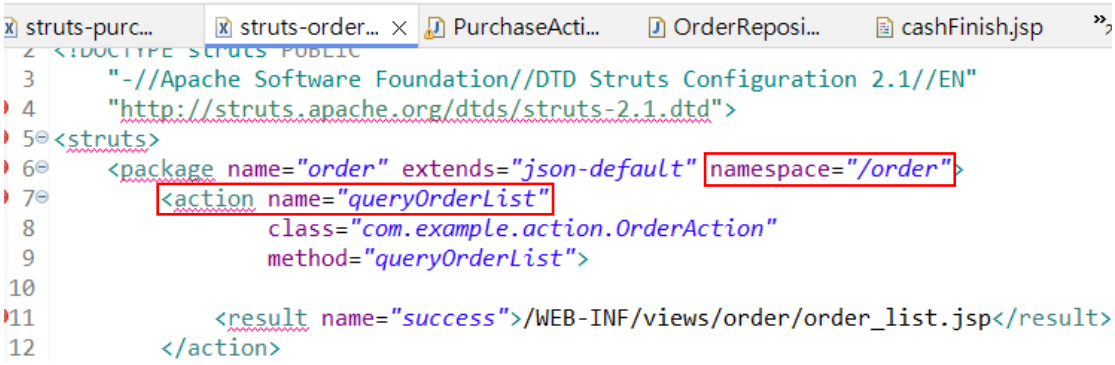


9.7.1 「查看訂單」鈕：

這時可以點擊「查看訂單」鈕，這樣即可進入訂單清單，查看剛剛的訂單：

```
<div class="modal-footer">
  <a href="/eshop/home/index" class="btn btn-primary">返回首頁</a>
  <a href="/eshop/order/queryOrderList" class="btn btn-outline-secondary">查看訂單</a>
</div>
```

一樣透過 struts 設定檔，找出該 url 對應的資源：



可以看到，對應的資源，是 OrderAction 的 queryOrderList 方法。

OrderAction 的 queryOrderList 方法：

```java
public class OrderAction extends BaseAction{

    @Autowired
    private OrderService orderService;

    List<Order> orderList;
    List<OrderItems> orderGroupList;
    private String message;

    public String queryOrderList() {
        User user = (User)getSession().getAttribute(ConstantName.SESSION_USER);

        if(user.getUserType()==1) {
            orderList = orderService.queryOrder(user.getLoginId());
        }else {
            orderGroupList = orderService.queryAllOrder();
        }
        return SUCCESS;
    }
```

orderService：

```java
    public List<Order> queryOrder(String loginId){
        Customer customer = customerRepository.findByLoginId(loginId)
                .orElseThrow(() -> new RuntimeException("查無此用戶"));

        List<Order> result = orderRepository.findByCustomer_CustId(customer.getCustId());

        return result;
    }
```

orderRepository：

透過 Spring Data JPA，做出透過 Customer 這個 Entity 的 custId 屬性，來查詢訂單資料：

```java
public interface OrderRepository extends JpaRepository<Order, Integer> {

    void updateOrderState(@Param("ordNum") String ordNum, @Param("state") String state);

    List<Order> findByCustomer_CustId(Integer custId);
```

返回 jsp 頁面，也就是訂單清單頁面。

9.8 訂單清單頁面：

order_list.jsp：





可以看到剛剛生成的訂單，狀態為已付款；而這個 ui 是使用 bootstrap 風格的手風琴 ui，搭配 Struts tag，即可重複呈顯手風琴元素 ui，並在每一個 ui 內寫入一筆訂單資料；下圖是 jsp 頁面具體的寫法：

```
<%@ include file="../exampleNav.jspf" %>
<h2 class="my-2">訂單詳情</h2>

<div class="w-75 mx-auto">
    <div class="accordion" id="orderAccordion">
        <s:iterator value="orderList" var="order" status="st">
            <div class="accordion-item">
                <h2 class="accordion-header" id="heading<s:property value='%{#st.index}'/>">
                    <button class="accordion-button collapsed" type="button"
                            data-bs-toggle="collapse"
                            data-bs-target="#collapse<s:property value='%{#st.index}'/>"
                            aria-expanded="false"
                            aria-controls="collapse<s:property value='%{#st.index}'/>">
                        訂單編號：<s:property value="#order.ordNum"/> |
                        金額：$<s:property value="#order.amount"/> |
                        狀態：<s:property value="#order.state"/>
                    </button>
                </h2>
                <div id="collapse<s:property value='%{#st.index}'/>"
                     class="accordion-collapse collapse"
                     aria-labelledby="heading<s:property value='%{#st.index}'/>"
                     data-bs-parent="#orderAccordion">
                    <div class="accordion-body">
                        <p><strong>商品：</strong> <s:property value="#order.product.prodName"/></p>
                        <p><strong>數量：</strong> <s:property value="#order.ordQty"/></p>
                        <p><strong>顧客姓名：</strong> <s:property value="#order.customer.custName"/></
                    </div>
                </div>
            </div>
        </s:iterator>
    </div>
</div>
<div class="text-center mt-2">
   <a href="<%= request.getContextPath() %>/home/index" class="btn btn-secondary">
      返回首頁
    </a>
</div>
```

9.9 更新版訂單清單頁面(以前端方式添加分頁功能)：

後端程式不變，前端以 bootstrap、Vue.js、bootstrap-vue 套件來設置分頁功能：

首先，設定 Vue.js 要作用的 html 範圍：

```
<div id="app">
        <div class="w-75 mx-auto">
            <div class="accordion" id="orderAccordion">
```

設定 Vue 物件。

設定變數，之後就可讓該變數跟來自 bootstrap-vue 套件的當前頁碼進行綁定：

```
<script>

        new Vue({
              el: '#app',
              data(){
                   return{
                        currentPage:1,
                        totalItems:document.querySelectorAll(".accordion-
item").length,
                        itemsPerPage:10
                   };
              },
<b-pagination
         v-model="currentPage"
         :total-rows="totalItems"
         :per-page="itemsPerPage"
         :page-range="10"
         :next-text="'Next'"
         :prev-text="'Previous'"
         class="d-flex justify-content-center my-2"
     ></b-pagination>
```

當前頁碼可以隨時改變，故 data 內也有能動態變化的值，該值用於 methods 內定義的函數，若該值改變，且該函數若與 v-if、v-show 綁定，則會動態執行，因此就能根據函數執行結果來決定元素是否要呈顯。

這樣，就能決定要呈顯的內容，即可做出分頁效果。

```
              methods:{
                   showItem(index){
                        if(index>=(this.currentPage-1)*this.itemsPerPage&&
                                 index<this.currentPage*this.itemsPerPage){
                             return true;
                        }
                        return false;
                   }
              }
        });
    </script>
<div class="accordion-item" v-show="showItem(<s:property
```

```
value='%{#st.index}'/>)">
```

9.10 更新版訂單清單頁面(以後端方式添加分頁功能)：

使用 Spring Data JPA 來實作分頁功能。

9.10.1 DAO：

```
Page<Order> findByCustomer_CustId(Integer custId, Pageable pageable);
```

9.10.2 Service：

```
public Page<Order> queryOrder(String loginId,Pageable page){
        Page<Order> result =
orderRepository.findByCustomer_CustId(customer.getCustId(),page);

        return result;
    }
```

9.10.2.1 另外檢查 loginId，是否有對應的用戶：

```
Customer customer = customerRepository.findByLoginId(loginId)
            .orElseThrow(() -> new RuntimeException("查無此用戶"));
```

上面的 orElseThrow，有值的話會回傳 Optional 所包含的物件，沒值的話則需要拋出一個例外，這個例外要透過 supplier 來拋出，故傳入的參數就是 supplier。

那麼，為何 orElseThrow 的參數，必須要寫成 supplier？不可直接寫成 new RuntimeException(...)？因為如果直接寫成 RuntimeException(...)，會變成無論如何都會產生該物件，如果一切正常的話，這個物件就變成多餘的。相反，寫成 supplier，就變成懶加載，就是確定有問題時才會執行。

9.10.3 Action：

```
private int page;
private int size;
```

```
Pageable pageable = PageRequest.of(page-1, size,
Sort.by("ordNumId").descending());
orderPageList = orderService.queryOrder(user.getLoginId(),pageable);
```

page、size 都要有 setXXX，作為跟瀏覽器查詢字串，綁定的語法。
具體綁定的方法，是 Struts 2 接收到前端表單或查詢字串後，跟據 key 值，比方說叫做 XXX，找出對應的 setXXX，將其寫入最近的 Action 的 XXX 屬性。

9.10.4 瀏覽器 URL：

```
<%= request.getContextPath() %>/order/queryOrderList?page=1&size=10
```

10 管理者端頁面，處理訂單功能：

10.1 DAO：

```
List<Order> orders = orderRepository.findAll();
@Query("SELECT distinct o.ordNum, o.state FROM Order o")
List<Object> findStateAndOrderNum();
```

10.2 Service：

```
Map<String, List<Order>> grouped = orders.stream()
        .collect(Collectors.groupingBy(Order::getOrdNum)); // 依訂單編號分組

        List<OrderItems> result = new ArrayList<>();

        for (Map.Entry<String, List<Order>> entry : grouped.entrySet()) {
            result.add(new OrderItems(entry.getKey(), entry.getValue()));
        }

        return result;
```

以上程式，Collectors.groupingBy(Order::getOrdNum)
可以改寫成：Collectors.groupingBy(x->x.getOrdNum())

Collectors 工具類，為何就是 Java Collector API？因為 Collectors 工具類，實作 Collector 這個規格書。

Stream API 要用 Java Collector API，就必須呼叫 collect()。

要從 map 逐一取出 key value，為何必須用 map.entrySet()，不可直接用 map？因為 map 是沒有實作迭代功能的。

那麼，map 為何沒有實作迭代功能？這不是很常用的功能嗎？因為 map 本身可以對 key-value 迭代，也可以對 key、value 迭代，直接實作迭代的話，會造成混淆。

Entry 不能直接當成一個類的原因：因為 Entry 是 Map 內部的資料結構。

10.3 Action：

```
orderGroupList = orderService.queryAllOrder();
stateOrderNumList=orderService.findStateAndOrderNum();
getRequest().setAttribute("stateOrderNumList", stateOrderNumList);

return SUCCESS;
```

10.4 struts-order.xml：

```
<package name="order" extends="json-default" namespace="/order">
<result name="success" type="json"/>
```

前者負責啟動將 HttpResponse Body 轉成 json 格式之功能。
後者負責將某個具體的 HttpResponse Body 轉成 json 格式。

10.5 order_list_forAdmin.jsp：

10.5.1 呈顯訂單資料：

```
<s:iterator value="orderGroupList" var="orderGroup" status="st">

訂單編號：<s:property value="#orderGroup.ordNum"/>

<s:iterator value="#orderGroup.orders" var="order">
    <tr>
        <td><s:property value="#order.product.prodName"/></td>
        <td><s:property value="#order.ordQty"/></td>
        <td>$<s:property value="#order.product.prodPrice"/></td>
        <td>$<s:property value="#order.product.prodPrice *
#order.ordQty"/></td>
```

```
    </tr>
</s:iterator>
```

這個 orderGroupList，在 Struts action 有同名屬性，且也有 getXXX，那麼 Struts 2 tag 是去哪裡找到 orderGroupList，是在 Struts 2 內部的 valueStack。

為何`<s:iterator value="orderGroupList" var="orderGroup" status="st">`必須是 value 在前面，因為 Struts 2 必須先知道要迭代的對象。

`<s:property value="#order.product.prodPrice * #order.ordQty"/>`

這樣寫即可得出結果，因為上面其實是執行 order.getProduct().getProdPrice()，故可以同時找出型別，就可確認可以相乘

10.5.2 設置修改訂單狀態功能：

10.5.2.1 設置出貨按鈕是否可以按：

```
<!-- 出貨按鈕 -->
<form class="mt-3 shipForm">
    <input type="hidden" name="ordNum" value="<s:property
value='#orderGroup.ordNum'/>">
    <input type="hidden" name="state" value="已出貨">

    <s:set var="ordNum" value="#orderGroup.ordNum" scope="request"/>
    <c:forEach var="stateOrderNum" items="${stateOrderNumList}">
        <c:choose>
            <c:when test="${stateOrderNum[0]==ordNum}">

                <c:choose>
                    <c:when test="${stateOrderNum[1]=='已出貨'}">
                    <button type="button" class="btn btn-secondary"
disabled>已出貨</button>
                    </c:when>
                    <c:when test="${stateOrderNum[1]=='已付款'}">
                    <button type="submit" class="btn btn-success">訂單出貨
</button>
                    </c:when>
                </c:choose>
            </c:when>
```

```
            <c:when test="${stateOrderNum[0]!=ordNum}">
            </c:when>
        </c:choose>
    </c:forEach>
</form>
```

上面為何是用 c:choose，而非 c:if？因為 c:choose 可以做到 if、else if 的效果，而 c:if 不行。

```
<s:set var="ordNum" value="#orderGroup.ordNum" scope="request"/>
```

跟

```
<s:set value="#orderGroup.ordNum" var="ordNum" scope="request"/>
```

都是可以的，因為這僅僅只是設定變數；但最好還是用後者的語法。

```
<c:forEach var="stateOrderNum" items="${stateOrderNumList}">
```

上面的 JSTL Tag，var 跟 value，順序跟 Struts tag 不同，因為 JSTL Tag，其語法是模擬 Java 傳統的 foreach 迴圈。

10.5.2.2 設置出貨鈕被按下後，要執行的動作：

```
<script>
        let lastClickedButton = null;

      $(document).ready(function () {
        $('.shipForm').on('submit', function (e) {

          e.preventDefault(); // 阻止表單預設提交行為
          lastClickedButton = $(this).find('button[type="submit"]');

          $.ajax({
            url:
'<%=request.getContextPath()%>/order/updateOrderStateByOrdNum',
            method: 'POST',
            //data: $(this).serialize(),

            data:new FormData(this),
            processData: false,
            contentType: false,

            success: function (response) {
              if(response.message=='訂單已成功出貨！'){
```

```
                const modal = new
bootstrap.Modal(document.getElementById('shipSuccessModal'));
                    modal.show();
                }
            },
            error: function () {
              alert("出貨失敗，請稍後再試。");
            }
          });
        });

        $('#modalCloseBtn').on('click', function () {
            lastClickedButton
              .text('已出貨')
              .removeClass('btn-success')
              .addClass('btn-secondary')
              .prop('disabled', true);
         })
      });
    </script>
```

```
$('.shipForm').on('submit', function (e) {
```

注意是表單觸發 submit 事件，而非提交按鈕來觸發，因此 this 指的就是表單物件。

```
data: $(this).serialize()
```

$(this).serialize()，表示對表單資料做序列化成 key-value 對，以便傳遞。

```
success: function (response) {
```

response 變數，這邊是 httpResponse Body，物件化的樣子。

```
response.message=='訂單已成功出貨！'
```

物件化的 httpResponse Body，內部內容就是 json 格式的 key-value，唯一一個 key 就是 message；這是在 Struts 2 的某個 Action 添加的。

```
new bootstrap.Modal(document.getElementById('shipSuccessModal'))
```

DOM 元素不可能直接具備 modal 的功能，所以 bootstrap 特別設計 modal 方法，給 DOM 元素裝飾成 modal 物件。

```
prop('disabled', true)
```

可以改用 attr？attr 操作 dom 元素的靜態屬性、prop 操作 dom 元素的動態屬性。

10.5.3 實際頁面：







這樣操作結束後，回到用戶頁面查看，就可看到訂單已被改成已出貨：



11 錯誤處理：

11.1 出現 java.lang.NoSuchMethodException: com.sun.proxy.$Proxy83.doLogin() 但是 com.example.action.LoginAction 確實有 doLogin 方法。

不要在 Action 上做 AOP。

11.2 debug 模式看到的物件值是 null，但是用 print 印出來又有值。

因為 DTO 用了 Spring AOP，Debug 模式看到的 DTO 是代理物件，不是真實物件，而代理物件沒有屬性值；但是如果呼叫 print，代理物件又會轉呼叫真實物件 所以 print 才會看到屬性值。